



Федеральное государственное автономное образовательное

учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное программное обеспечение

Дисциплина «Основы профессиональной деятельности»

## **Отчёт по лабораторной работе №7**

### **Синтез команд БЭВМ**

Вариант №15517

Выполнил

Галак Екатерина Анатольевна

Р3115

Проверил

Блохина Елена Николаевна

Санкт – Петербург, 2025

## Задание

Синтезировать цикл исполнения для выданных преподавателем команд. Разработать тестовые программы, которые проверяют каждую из синтезированных команд. Загрузить в микропрограммную память БЭВМ циклы исполнения синтезированных команд, загрузить в основную память БЭВМ тестовые программы. Проверить и отладить разработанные тестовые программы и микропрограммы.

Введите номер варианта

1. XORSP - Исключающее ИЛИ двух верхних чисел на вершине стека, результат поместить на стек, установить признаки N/Z
2. Код операции - 0F02
3. Тестовая программа должна начинаться с адреса 03F6<sub>16</sub>

### Исходный код синтезируемой команды

| Адрес ячейки | Микрокоманда | Комментарий (расшифровка) | Комментарий                                                                                                                              |
|--------------|--------------|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| E0           | 0080009208   | ~0 + SP -> AR             | Сохраняем AC для восстановления после завершения выполнения целевой команды                                                              |
| E1           | 0001009010   | AC -> DR                  |                                                                                                                                          |
| E2           | 0200000000   | DR -> MEM(AR)             |                                                                                                                                          |
| E3           | 0080009008   | SP -> AR                  | Получаем значение первого числа из стека. Результат кладем в BR                                                                          |
| E4           | 0100000000   | MEM(AR) -> DR             |                                                                                                                                          |
| E5           | 0020009001   | DR -> BR                  |                                                                                                                                          |
| E6           | 0080009408   | SP + 1 -> AR              | Получаем значение второго числа из стека. Результат кладем в DR                                                                          |
| E7           | 0100000000   | MEM(AR) -> DR             |                                                                                                                                          |
| E8           | 0010009921   | BR & ~DR -> AC            | Выполняем XOR двух первых чисел, лежащих в стеке (первое лежит в BR, второе – в DR). Результат записываем в BR. Устанавливаем флаги N, Z |
| E9           | 0001009A21   | ~BR & DR -> DR            |                                                                                                                                          |
| EA           | 0020009B11   | ~AC & ~DR -> BR           |                                                                                                                                          |
| EB           | 0020809220   | ~BR -> BR; N, Z           | Восстанавливаем значение AC. Сразу подготавливаем SP к записи результата                                                                 |
| EC           | 0088009208   | ~0 + SP -> SP, AR         |                                                                                                                                          |
| ED           | 0100000000   | MEM(AR) -> DR             |                                                                                                                                          |
| EE           | 0010009001   | DR -> AC                  | Запись результата в стек                                                                                                                 |
| EF           | 0001009020   | BR -> DR                  |                                                                                                                                          |
| F0           | 0200000000   | DR -> MEM(AR)             | Переход к циклу прерывания                                                                                                               |
| F1           | 80C4101040   | GOTO INT @ C4             |                                                                                                                                          |

LD #05  
PUSH  
NEG  
PUSH  
NEG  
INC  
WORD 0x0F02  
HLT

## Таблица трассировки микропрограммы

В стеке находятся числа 0x37A0 и 0xB01F, в AC находится значение 0xB01F

| MP до<br>выборки<br>МК | Содержимое памяти и регистров процессора после выборки микрокоманды |     |      |     |      |     |      |      |      |              |
|------------------------|---------------------------------------------------------------------|-----|------|-----|------|-----|------|------|------|--------------|
|                        | MR                                                                  | IP  | CR   | AR  | DR   | SP  | BR   | AC   | NZVC | СчМК<br>(MP) |
| E0                     | 0080009208                                                          | 109 | 0F02 | 7FC | 0F02 | 7FD | 108  | B01F | 1000 | E1           |
| E1                     | 0001009010                                                          | 109 | 0F02 | 7FC | B01F | 7FD | 108  | B01F | 1000 | E2           |
| E2                     | 0200000000                                                          | 109 | 0F02 | 7FC | B01F | 7FD | 108  | B01F | 1000 | E3           |
| E3                     | 0080009008                                                          | 109 | 0F02 | 7FD | B01F | 7FD | 108  | B01F | 1000 | E4           |
| E4                     | 0100000000                                                          | 109 | 0F02 | 7FD | B01F | 7FD | 108  | B01F | 1000 | E5           |
| E5                     | 0020009001                                                          | 109 | 0F02 | 7FD | B01F | 7FD | B01F | B01F | 1000 | E6           |
| E6                     | 0080009408                                                          | 109 | 0F02 | 7FE | B01F | 7FD | B01F | B01F | 1000 | E7           |
| E7                     | 0100000000                                                          | 109 | 0F02 | 7FE | 37A0 | 7FD | B01F | B01F | 1000 | E8           |
| E8                     | 0010009921                                                          | 109 | 0F02 | 7FE | 37A0 | 7FD | B01F | 801F | 1000 | E9           |
| E9                     | 0001009A21                                                          | 109 | 0F02 | 7FE | 07A0 | 7FD | B01F | 801F | 1000 | EA           |
| EA                     | 0020009B11                                                          | 109 | 0F02 | 7FE | 07A0 | 7FD | 7840 | 801F | 1000 | EB           |
| EB                     | 0020809220                                                          | 109 | 0F02 | 7FE | 07A0 | 7FD | 87BF | 801F | 1000 | EC           |
| EC                     | 0088009208                                                          | 109 | 0F02 | 7FC | 07A0 | 7FC | 87BF | 801F | 1000 | ED           |
| ED                     | 0100000000                                                          | 109 | 0F02 | 7FC | B01F | 7FC | 87BF | 801F | 1000 | EE           |
| EE                     | 0010009001                                                          | 109 | 0F02 | 7FC | B01F | 7FC | 87BF | B01F | 1000 | EF           |
| EF                     | 0001009020                                                          | 109 | 0F02 | 7FC | 87BF | 7FC | 87BF | B01F | 1000 | F0           |
| F0                     | 0200000000                                                          | 109 | 0F02 | 7FC | 87BF | 7FC | 87BF | B01F | 1000 | F1           |
| F1                     | 80C4101040                                                          | 109 | 0F02 | 7FC | 87BF | 7FC | 87BF | B01F | 1000 | C4           |

## Тестовая программа

```

ORG 0x3EC
RES: WORD 0x1
RES_TEST_1: WORD ?
RES_TEST_2: WORD ?
RES_TEST_3: WORD ?
RES_TEST_4: WORD ?
RES_TEST_5: WORD ?
RES_TEST_6: WORD ?

ORG 0x3F6
START:
    CALL $TEST_1                ; вызов первого теста
    LD $RES_TEST_1
    AND $RES
    ST $RES
    HLT

    CALL $TEST_2                ; вызов второго теста
    LD $RES_TEST_2
    AND $RES
    ST $RES
    HLT

    CALL $TEST_3                ; вызов третьего теста
    LD $RES_TEST_3
    AND $RES
    ST $RES
    HLT

```

```

CALL $TEST_4 ; вызов четвертого теста
LD $RES_TEST_4
AND $RES
ST $RES
HLT

CALL $TEST_5 ; вызов пятого теста
LD $RES_TEST_5
AND $RES
ST $RES
HLT

CALL $TEST_6 ; вызов шестого теста
LD $RES_TEST_6
AND $RES
ST $RES
HLT

LD $RES

HLT

; проверка корректности работы команды XORSP
ORG 0x100
X_1: WORD 0x37A0 ; \
X_2: WORD 0xB01F ; -> xor = 87BF
MY_RES_TEST_1: WORD 0x87BF

TEST_1:
LD $X_1
PUSH
LD $X_2
PUSH
WORD 0x0F02 ; команда XORSP
NOP
POP ; забираем результат выполнения команды со стека
CMP MY_RES_TEST_1 ; сравниваем с результатом, посчитанным вручную
BNE ERROR_TEST_1 ; если результат не сошелся, записываем ошибку
POP ; снимаем со стека второе число (X_2)
POP ; снимаем со стека первое число (X_1)
LD #0x01
ST $RES_TEST_1
RET

ERROR_TEST_1:
POP ; снимаем со стека второе число (X_2)
POP ; снимаем со стека первое число (X_1)
LD #0x00
ST $RES_TEST_1 ; при некорректном результате записываем ноль
RET

; проверка корректности сохранения AC (AC должен остаться равным значению,
которое было до выполнения команды)
ORG 0x125
X_3: WORD 0x6719 ; \
X_4: WORD 0x0043 ; -> xor = 675A
START_AC: WORD 0x109

TEST_2:
LD $X_3

```

```

    PUSH
    LD $X_4
    PUSH
    LD $START_AC
    WORD 0x0F02 ; команда XORSP
    NOP
    CMP $START_AC ; сравниваем с результатом, посчитанным вручную
    BNE ERROR_TEST_2 ; если результат не сошелся, записываем ошибку
    POP ; забираем результат выполнения команды со стека
    POP ; снимаем со стека второе число (X_4)
    POP ; снимаем со стека первое число (X_3)
    LD #0x01
    ST $RES_TEST_2
    RET

ERROR_TEST_2:
    POP ; забираем результат выполнения команды со стека
    POP ; снимаем со стека второе число (X_4)
    POP ; снимаем со стека первое число (X_3)
    LD #0x00
    ST $RES_TEST_2 ; при некорректном результате записываем ноль
    RET

; проверка корректности установки признака Z (Z = 0)
ORG 0x150
X_5: WORD 0x813C ;\
X_6: WORD 0x76AA ; -> xor = F796

TEST_3:
    LD $X_5
    PUSH
    LD $X_6
    PUSH
    WORD 0x0F02 ; команда XORSP
    NOP
    BEQ ERROR_TEST_3 ; если результат не сошелся, записываем ошибку
    (BEQ - переход, если Z = 1, в нашем случае Z должен быть равен нулю)
    POP ; забираем результат выполнения команды со стека
    POP ; снимаем со стека второе число (X_6)
    POP ; снимаем со стека первое число (X_5)
    LD #0x01
    ST $RES_TEST_3
    RET

ERROR_TEST_3:
    POP ; забираем результат выполнения команды со стека
    POP ; снимаем со стека второе число (X_6)
    POP ; снимаем со стека первое число (X_5)
    LD #0x00
    ST $RES_TEST_3 ; при некорректном результате записываем ноль
    RET

; проверка корректности установки признака Z (Z = 1)
ORG 0x175
X_7: WORD 0x1498 ;\
X_8: WORD 0x1498 ; -> xor = 0

TEST_4:
    LD $X_7
    PUSH
    LD $X_8
    PUSH
    WORD 0x0F02 ; команда XORSP

```

```

    NOP
    BNE ERROR_TEST_4          ; если результат не сошелся, записываем ошибку
(BNE - переход, если Z = 0, в нашем случае Z должен быть равен единице)
    POP                      ; забираем результат выполнения команды со стека
    POP                      ; снимаем со стека второе число (X_8)
    POP                      ; снимаем со стека первое число (X_7)
    LD #0x01
    ST $RES_TEST_4
    RET

ERROR_TEST_4:
    POP                      ; забираем результат выполнения команды со стека
    POP                      ; снимаем со стека второе число (X_8)
    POP                      ; снимаем со стека первое число (X_7)
    LD #0x00
    ST $RES_TEST_4          ; при некорректном результате записываем ноль
    RET

; проверка корректности установки признака N (N = 1)
ORG 0x200
X_9: WORD 0xABE7 ; \
X_10: WORD 0x1F08 ; -> xor = B4EF

TEST_5:
    LD $X_9
    PUSH
    LD $X_10
    PUSH
    WORD 0x0F02             ; команда XORSP
    NOP
    BPL ERROR_TEST_5        ; если результат не сошелся, записываем ошибку
(BPL - переход, если N = 0, в нашем случае N должен быть равен единице)
    POP                      ; забираем результат выполнения команды со стека
    POP                      ; снимаем со стека второе число (X_10)
    POP                      ; снимаем со стека первое число (X_9)
    LD #0x01
    ST $RES_TEST_5
    RET

ERROR_TEST_5:
    POP                      ; забираем результат выполнения команды со стека
    POP                      ; снимаем со стека второе число (X_10)
    POP                      ; снимаем со стека первое число (X_9)
    LD #0x00
    ST $RES_TEST_5          ; при некорректном результате записываем ноль
    RET

; проверка корректности установки признака N (N = 0)
ORG 0x225
X_11: WORD 0x56E1 ; \
X_12: WORD 0x467F ; -> xor = 109E

TEST_6:
    LD $X_11
    PUSH
    LD $X_12
    PUSH
    WORD 0x0F02             ; команда XORSP
    NOP
    BMI ERROR_TEST_6        ; если результат не сошелся, записываем ошибку
(BMI - переход, если N = 1, в нашем случае N должен быть равен нулю)
    POP                      ; забираем результат выполнения команды со стека
    POP                      ; снимаем со стека второе число (X_12)

```

```

POP                                ; снимаем со стека первое число (X_11)
LD #0x01
ST $RES_TEST_6
RET

ERROR_TEST_6:
POP                                ; забираем результат выполнения команды со стека
POP                                ; снимаем со стека второе число (X_12)
POP                                ; снимаем со стека первое число (X_11)
LD #0x00
ST $RES_TEST_6                    ; при некорректном результате записываем ноль
RET

```

## Описание тестов

1) Первый тест проверяет корректность работы выполнения операции XOR между двух верхних чисел в стеке.

2) Второй тест проверяет корректность сохранения АС после выполнения программы (внутри синтезируемой программы используется АС, после завершения её выполнения необходимо вернуть значение, находившееся в АС до начала выполнения команды).

3) Третий тест проверяет корректность установки признака Z = 0.

4) Четвертый тест проверяет корректность установки признака Z = 1.

5) Пятый тест проверяет корректность установки признака N = 1.

6) Шестой тест проверяет корректность установки признака N = 0.

## Методика проверки команды с использованием тестовой программы

1) Запустить БЭВМ в режиме Dual (для этого используем команду `java -Dmode=dual -jar bcomp-ng.jar`).

2) Вводим последовательность команд в терминал (производим загрузку микрокоманд в память микрокоманд)

```

ma
mw 0080009208
mw 0001009010
mw 0200000000
mw 0080009008
mw 0100000000
mw 0020009001
mw 0080009408
mw 0100000000
mw 0010009921
mw 0001009A21
mw 0020009B11
mw 0020809220
mw 0088009208

```



```
mw 0100000000  
mw 0010009001  
mw 0001009020  
mw 0200000000  
mw 80C4101040  
mdecodeall
```

3) Загружаем тестовую программу в БЭВМ и запускаем её. Дожидаемся останова, если в АС в этот момент записана 1, тест пройден успешно (если 0 – произошла ошибка). Также результат каждого теста записывается в отдельную переменную в память (RES\_TEST\_N, где N – номер теста, в памяти находится с 0x3ED до 0x3F1). Если все тесты пройдены корректно, в АС и в переменной RES (0x3EC) будет записана 1.

## **Заключение**

Во время выполнения лабораторной работы я научилась синтезировать команды для БЭВМ, тестировать работу микропрограммы и выполнять их трассировку.